

Politechnika Rzeszowska im. Ignacego Łukasiewicza

Wydział Matematyki i Fizyki Stosowanej



WYDZIAŁ
MATEMATYKI
I FIZYKI STOSOWANEJ
POLITECHNIKI RZESZOWSKIEJ

**Generowanie odpowiedzi na wiadomości e-mail za pomocą
generatywnych modeli językowych**

st. FSO-DI, gr. L2 Paweł Górski, Mateusz Gałda

Rzeszów, 2025

Spis treści

1. Wprowadzenie.....	4
2. Sformułowanie zadania projektowego	4
2.1 Wprowadzenie do tworzenia dodatków w Thunderbird	4
2.2 API ChatGPT.....	5
3. Rozpoczęcie prac	5
3.1 Dodatek do Thunderbirda	5
3.1.1 Problemy z Thunderbirdem.....	6
3.1.2 Problemy z protokołem CORS i API od ChatGPT	7
4. Zmiana podejścia i wykorzystanie API od Google Gemini	8
4.1 Zestawienie tabelaryczne API Gemini i API ChatGPT	9
4.2 Testy API od Google Gemini	10
4.2 Budowa aplikacji do generowania odpowiedzi	11
4.3 Tworzenie topologii komunikacji między klientem, serwerem Flask i modelem Google Gemini	14
5. Podsumowanie oraz określenie dalszych kierunków rozwoju aplikacji.....	15
5.1 Podsumowanie	16
6. Bibliografia.....	16

1. Wprowadzenie

W dobie rosnącej cyfryzacji i automatyzacji, jednym z najważniejszych wyzwań współczesnych organizacji jest efektywne zarządzanie komunikacją elektroniczną, szczególnie w kontekście zarządzania pocztą elektroniczną. Szybko rosnąca liczba wiadomości e-mail, zarówno w życiu zawodowym, jak i prywatnym, wymaga coraz bardziej zaawansowanych narzędzi do ich obsługi. W odpowiedzi na tę potrzebę, rozwój technologii opartych na sztucznej inteligencji, w tym generatywnych modeli językowych, stwarza nowe możliwości automatyzacji odpowiedzi na e-maile. Często wiadomości e-mail, które otrzymujemy okazują się być spamem lub po prostu nas nie interesują, jednak dla zachowania profesjonalizmu lub nawet dobrych relacji pomiędzy firmami lub instytucjami powinniśmy odpowiadać na nie.

2. Sformułowanie zadania projektowego

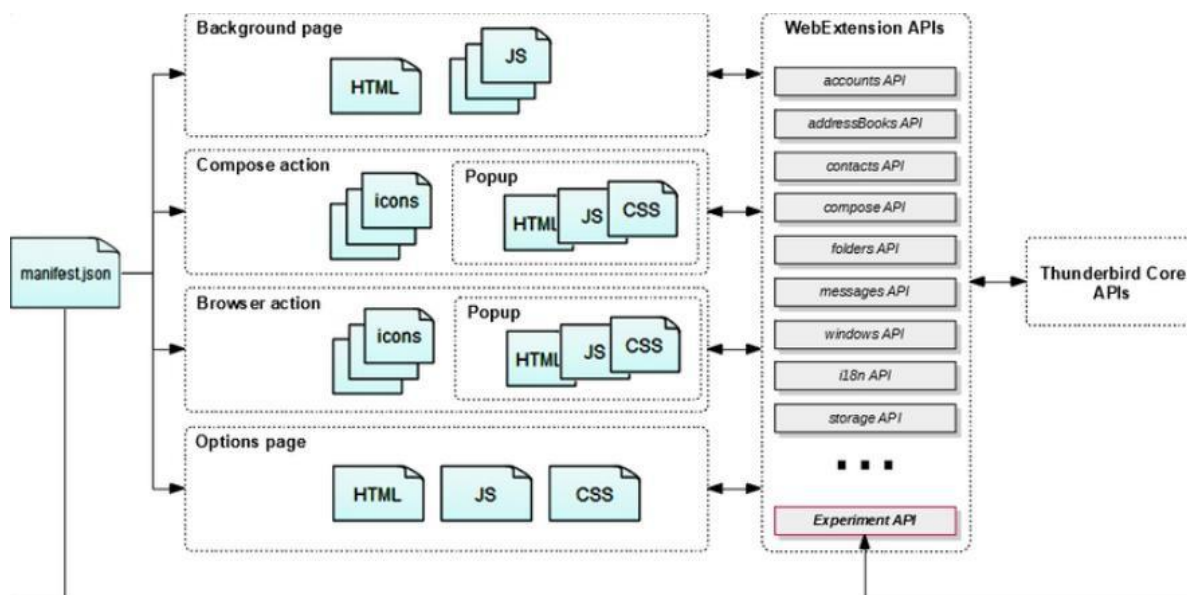
Celem niniejszej pracy jest stworzenie aplikacji zintegrowanej z klientem poczty, który będzie wykorzystywał modele generatywne do automatycznego generowania odpowiedzi na wiadomości e-mail, które zawierają zapytania, propozycje lub oferty. Użytkownik będzie miał możliwość wyboru zaakceptowania lub odrzucenia danej propozycji, a wiadomość będzie mogła zostać wygenerowana w stylu formalnym lub nieformalnym - w zależności od wyboru użytkownika. Aplikacja będzie podłączona do Thunderbirda w formie dodatku.

2.1 Wprowadzenie do tworzenia dodatków w Thunderbird

Obecne dodatki do Thunderbirda opierają się na technologii WebExtension, która jest również szeroko wykorzystywana przez inne przeglądarki internetowe. Dostawcy przeglądarek często nazywają swoje rozszerzenia WebExtensions. Jednakże, Thunderbird oferuje funkcje, które nie są dostępne w przeglądarkach, przez co dodatki stworzone dla Thunderbirda raczej nie będą działały w innych przeglądarkach. Z tego względu nazywane są one MailExtensions.

WebExtensions to zestaw plików (rys. 1), które zmieniają wygląd i funkcjonalność Thunderbirda. Mogą dodawać elementy interfejsu użytkownika, modyfikować zawartość lub wykonywać zadania w tle. Dodatki do Thunderbirda są tworzone z wykorzystaniem standardowego JavaScriptu, CSS i HTML¹.

¹ <https://developer.thunderbird.net/add-ons/about-add-ons>



Rysunek 1. Struktura dodatków dla Thunderbird

2.2 API ChatGPT

API (Interfejs Programowania Aplikacji) to zbiór zasad i protokołów, które umożliwiają komunikację pomiędzy różnymi aplikacjami, w tym w czasie rzeczywistym. Dzięki temu narzędziu, jeden program może wykorzystywać funkcje innego, nie musząc znać jego wewnętrznej struktury, a wymiana danych między aplikacjami staje się możliwa.²

Aby korzystać z API, należy uzyskać unikalny klucz dostępu, który pozwala na autoryzowany dostęp do platformy. Komunikacja z API odbywa się poprzez wysyłanie zapytań do odpowiednich endpointów – adresów URL, które dają nam dostęp do korzystania z funkcji drugiego programu. W naszym przypadku korzystamy z endpointu <https://api.openai.com/v1/completions>, który będzie generował odpowiedź na podstawie przekazanych zapytań.

3. Rozpoczęcie prac

3.1 Dodatek do Thunderbirda

Dodatek składa się z pięciu kluczowych plików: *background.js*, *popup.html*, *config.json*, *manifest.json* oraz *popup.js*. Każdy z nich pełni określoną funkcję, współpracując w celu

² <https://coderslab.pl/pl/blog/co-to-jest-api-wszystko-o-interfejsie-programowania-aplikacji>

zapewnienia prawidłowego działania dodatku. Razem tworzą mechanizm umożliwiający generowanie odpowiedzi na e-maile w oparciu o wybrane przez użytkownika parametry.

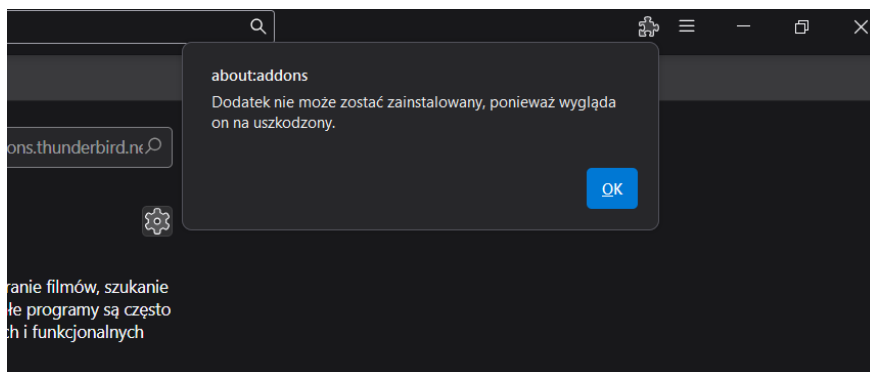
Plik *background.js* zarządza działaniami, które nie wymagają bezpośredniej interakcji użytkownika z interfejsem. Jego zadaniem jest obsługa zdarzeń i reagowanie na akcje. W naszym przypadku wykorzystaliśmy go w celu generowania odpowiedzi na e-maile na podstawie wprowadzenia użytkownika. Przyjmuje on trzy wartości: treść e-maila, styl odpowiedzi oraz typ odpowiedzi na daną propozycję (zgoda lub odmowa). Na podstawie tych danych funkcja tworzy zapytanie typu POST, które służy do wysłania danych od klienta na serwer za pomocą endpointu.

Plik *popup.html* odpowiada za interfejs dodatku. To on odpowiada za panel sterowania do generowania odpowiedzi. Użytkownik wybiera parametry, które następnie zostają przetwarzane i wysyłane przez *background.js*.

Pliki takie jak *config.json*, *manifest.json* oraz *popup.js* pełnią funkcje pomocnicze. Plik *config.json* przechowuje ustawienia konfiguracyjne, podczas gdy *manifest.json* jest niezbędny do zdefiniowania struktury dodatku w Thunderbirdzie i jego integracji z systemem, natomiast *popup.js* zarządza logiką interakcji z elementami interfejsu w oknie popup.

3.1.1 Problemy z Thunderbirdem

Po stworzeniu wszystkich wymaganych plików przeszliśmy do próby dołączenia plików do Thunderbirda. Niestety napotkaliśmy problem od strony klienta poczty, który komunikował, że jeden z plików jest uszkodzony (rys. 2). Nie dysponowaliśmy wystarczającą ilością zasobów oraz czasu, aby rozwiązać ten problem.



Rysunek 2 Widok z Thunderbird

3.2 Stworzenie aplikacji serwerowej z wykorzystaniem XAMPP

Liczne problemy techniczne związane z tworzeniem dodatku do Thunderbirda zmusiły nas do zmiany podejścia. Postanowiliśmy stworzyć aplikację, w której XAMPP pełniłby rolę lokalnego serwera, komunikującego się z modelem sztucznej inteligencji w celu generowania odpowiedzi. Aplikacja została napisana w języku PHP i osadzona na serwerze Apache, który jest integralną częścią XAMPP.

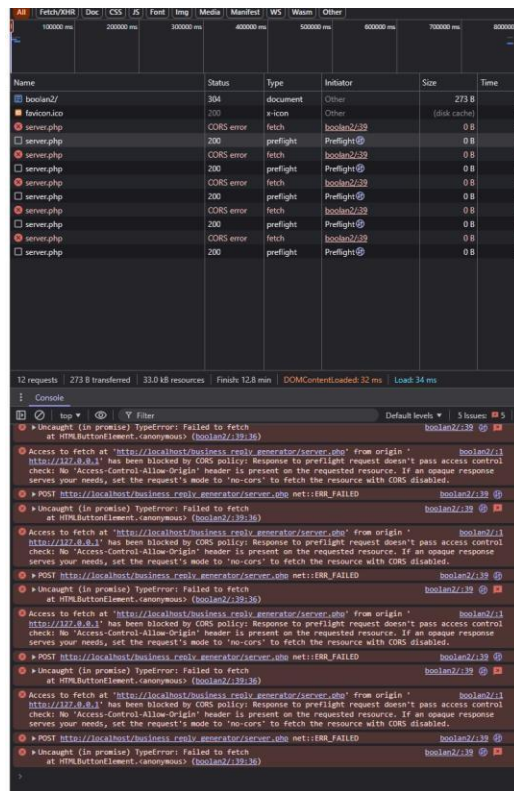
Aplikacja miała działać w ten sposób, że użytkownik wyśle żądanie HTTP metodą POST do serwera lokalnego, przesyłając dane w formacie JSON. Dane te powinny zawierać decyzję, styl odpowiedzi oraz treść oryginalnego e-maila. Serwer przyjmie dane, stworzy odpowiedni komunikat, a następnie wyśle go do interfejsu API OpenAI w celu wygenerowania odpowiedzi. Po otrzymaniu odpowiedzi od modelu GPT, serwer zwróci ją do użytkownika w formacie JSON. Jako interfejs użytkownika stworzyliśmy prosty frontend w HTML.

3.1.2 Problemy z protokołem CORS i API od ChatGPT

Podczas próby uzyskania odpowiedzi napotkaliśmy problemy. Pierwszym z nich stanowił protokół CORS (Cross-Origin Resource Sharing). Jest to zaawansowany mechanizm bezpieczeństwa stron internetowych, którego zadaniem jest poinformowanie przeglądarki, czy klientowi mogą zostać udostępnione dane związane z jego żądaniem. Realizowane jest to poprzez dodatkowe nagłówki HTTP. Ważne jest, że zapytanie jest zawsze poprawnie wykonywane. Nawet jeśli brak jest zgody na zwrot danych, odpowiedź serwera jest generowana, natomiast to CORS decyduje, czy dane zostaną faktycznie zwrócone³. Problem ten wystąpił, ponieważ nasza aplikacja próbowała komunikować się z lokalnym serwerem z poziomu przeglądarki, co domyślnie zostało zablokowane jako żądanie pochodzące z innego źródła.

³ <https://jakwybrachosting.pl/cors/>

- Komunikat protokołu CORS (rys. 3)



Rysunek 3 Widok z konsoli przeglądarki

Drugim problemem, który uniemożliwił dalsze korzystanie z narzędzia od OpenAI, był wyczerpany limit środków na koncie, na którym wygenerowano klucz API. Początkowo zakładaliśmy, że OpenAI oferuje darmowy limit w wysokości 5 USD na korzystanie z narzędzi, jednak środki te były dostępne wyłącznie przez pierwsze trzy miesiące od założenia konta, które posiadaliśmy już od pewnego czasu. Po upływie tego okresu, dostęp do API wymagał uzupełnienia środków, co uniemożliwiło nam kontynuację testów i pracy nad projektem.

4. Zmiana podejścia i wykorzystanie API od Google Gemini

Po konsultacjach postanowiliśmy zmienić sposób stworzenia generatora odpowiedzi. Zamiast tworzenia dodatku do klienta poczty postanowiliśmy stworzyć aplikację w Pythonie z wykorzystaniem biblioteki Flask. Jest to popularny mikroframework do tworzenia aplikacji webowych. Mikroframeworki charakteryzują się tym, że oferują tylko niezbędne funkcje, co umożliwia szybkie tworzenie prostych aplikacji bez zbędnego obciążenia. Flask wyróżnia się prostotą i elastycznością, dzięki czemu można go łatwo dostosować do indywidualnych

potrzeb. Framework ten oparty jest na protokole HTTP i dostarcza wiele narzędzi, takich jak wbudowany serwer, które ułatwiają proces tworzenia aplikacji webowych⁴. Biblioteka Flask pozwoliła nam na wyrenderowanie strony HTML, która odpowiada za interfejs użytkownika, oraz dostęp do serwera, który pobiera informacje od użytkownika, wysyła zapytania POST do serwera modelu sztucznej inteligencji, który generuje odpowiedź, która jest widoczna na stronie HTML.

Zmuszeni brakiem środków na korzystanie z narzędzi od firmy OpenAI postanowiliśmy zmienić model, który generuje odpowiedzi. Wybraliśmy model Gemini 1.5 Flash od firmy Google. Główną zaletą wybranego modelu jest darmowy plan bez ograniczenia czasowego. Posiada on również bogatą dokumentację oraz gotowe biblioteki w języku Python. Poniżej znajduje się porównanie produktów od Google i OpenAI.

4.1 Zestawienie tabelaryczne API Gemini i API ChatGPT

W tabeli 1 przedstawiono porównanie wybranych cech Gemini i ChatGPT.

Tabela 1 Charakterystyka porównawcza Gemini i ChatGPT

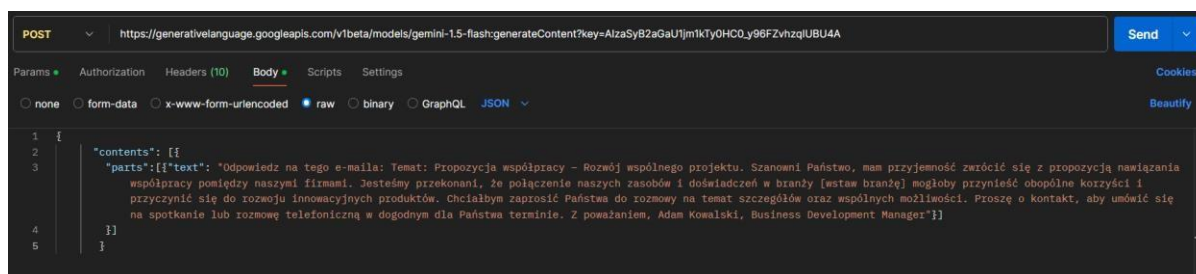
Cecha	Gemini 1.5 Flash	ChatGPT
Limit tokenów	15 RPM (żądań na minutę) 1 mln. TPM (tokenów na minutę) 1500 RPD (żądań dziennie)	3 RPM (żądań na minutę) 40 tys. TPM (tokenów na minutę) 200 RPD (żądań dziennie)
Cena	Plan darmowy	Brak planu darmowego, bezpłatny okres próbny
Integracja	Możliwość integracji z aplikacjami Google (np. Mapy, Gmail) w ramach darmowego dostępu	Brak możliwości integracji z aplikacjami ekosystemu Google w bezpłatnym wariantcie
Dokumentacja i informacje	Bogata dokumentacja, jasno opisane plany i modele	Mało dokumentacji o planach płatności

⁴ <https://boringowl.io/tag/flask>

4.2 Testy API od Google Gemini

Naszym priorytetem było otrzymanie odpowiedzi od API Gemini. W tym celu wykorzystaliśmy narzędzie Postman. Jest to platforma zaprojektowana do testowania, rozwijania i dokumentowania API. Narzędzie to umożliwia nie tylko tworzenie zapytań HTTP, ale także analizowanie odpowiedzi, co jest kluczowe dla zrozumienia, jak działają różne systemy i interfejsy⁵. Narzędzie to umożliwiło nam weryfikację działania klucza oraz sprawdzenie, czy otrzymujemy odpowiedzi na nasze zapytania POST. Skonfigurowaliśmy nasz endpoint: https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateContent?key=Klucz_api, który służy do generowania treści za pomocą modelu językowego Gemini 1.5 Flash. Treść, na którą model generatywny ma odpowiedzieć, jest przesyłana jako żądanie, używające format JSON w kluczu *contents*. W tym kluczu znajduje się lista zawierająca bloki danych z sekcją *parts* gdzie treść wiadomości zostaje umieszczona w polu *text*. Model Gemini 1.5 Flash analizuje przesłaną treść i generuje odpowiedź w czasie rzeczywistym, zwracając ją również w formacie JSON, w kluczu *generatedContent*.

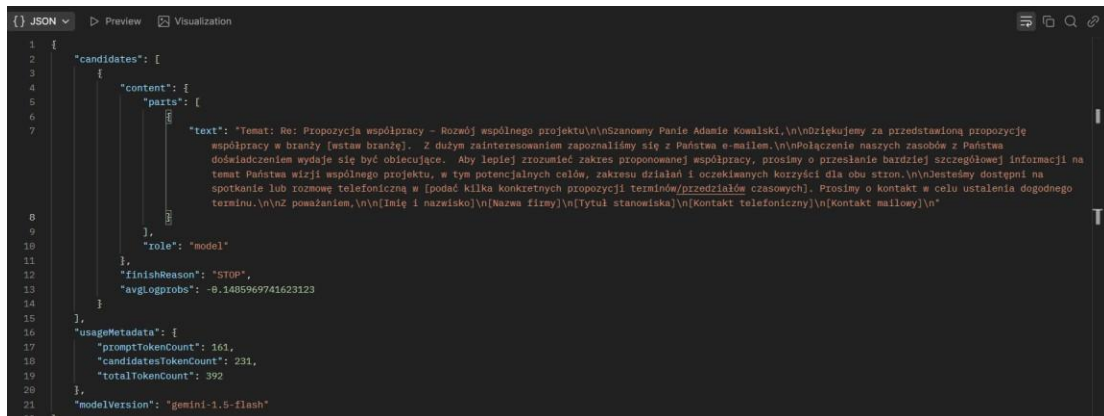
- Wysłanie żądania POST (rys. 4)



Rysunek 4 Widok z Postman1

⁵ <https://qualityisland.pl/postman-co-to-jest-jak-zainstalowac-i-uzywac-to-narzedzie/>

- Odpowiedź od Gemini (rys. 5)



Rysunek 5 Widok z Postman2

4.2 Budowa aplikacji do generowania odpowiedzi

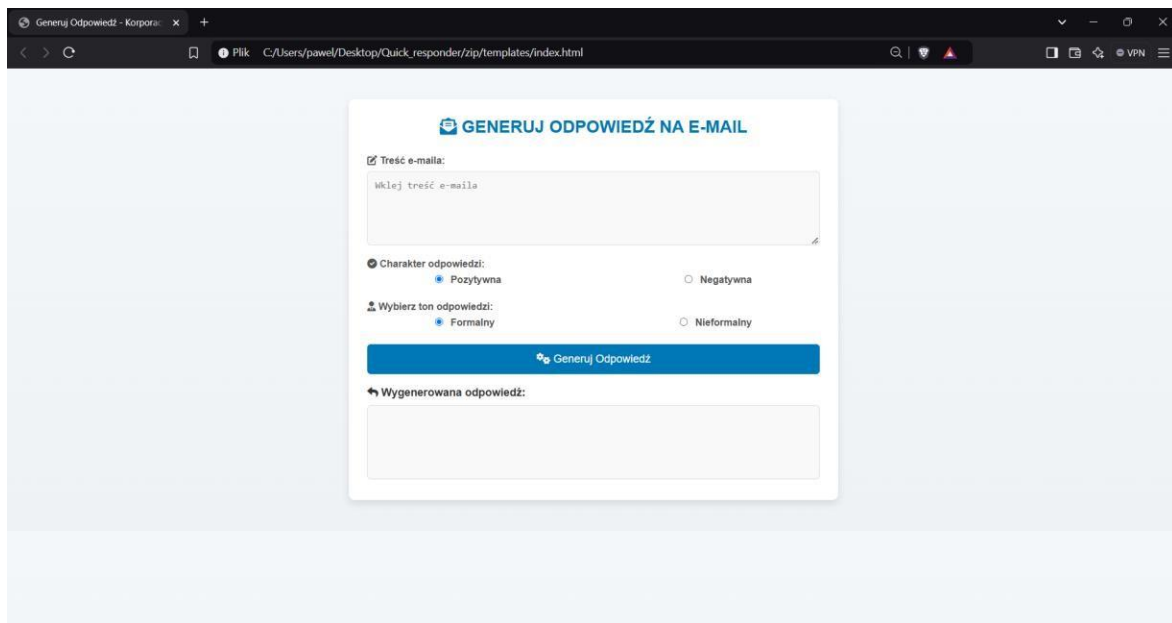
Aby rozwiązać problem z polityką CORS, wykorzystaliśmy rozszerzenie Flask-CORS, które pozwala na umożliwienie komunikacji między frontendem (interfejsem użytkownika), a backendem (serwerem). W naszym przypadku korzystamy z ustawienia, że wszystkie domeny mogą komunikować się z naszym serwerem na określonym endpointcie (*/generate_reply*), co oznacza, że każda strona internetowa może wysłać zapytanie do serwera.

Dane zostają pobierane z frontendu za pomocą zapytania HTTP POST. Gdy użytkownik wypełni formularz na stronie, wprowadzi treść odebranego e-maila, wybierze decyzję (tak lub nie) oraz styl odpowiedzi (formalny lub nieformalny), a po kliknięciu przycisku do wysłania formularza, dane zostaną przesyłane do backendu w formacie JSON. Backend odbiera te dane za pomocą funkcji `request.get_json()`, która umożliwia odczytanie przesłanych danych JSON.

Po odebraniu danych, backend przeprowadza ich walidację – sprawdza czy treść wiadomości e-mail nie jest pusta, czy decyzja ma wartość *tak* lub *nie*, oraz czy ton odpowiedzi jest formalny lub nieformalny. Jeżeli jedno z powyższych pól jest nieprawidłowe, serwer zwraca komunikat o błędzie w formacie JSON, z kodem odpowiedzi HTTP 400 (Bad Request), informując użytkownika o błędnych danych wejściowych. W przypadku poprawnych danych, backend tworzy zapytanie (prompt) do modelu sztucznej inteligencji Gemini, które jest wysyłane za pomocą funkcji `model.generate_content(prompt)`. Model generuje odpowiedź i zwraca ją do serwera, gdzie następnie zostaje przypisana do zmiennej *response*. Ostatecznie backend wysyła wygenerowaną odpowiedź z powrotem do frontendu w formacie JSON za pomocą funkcji `jsonify({'reply': response.text})`.

Strona w HTML tworzy frontend aplikacji. Strona zawiera formularz, w którym użytkownik wprowadza treść e-maila, wybiera decyzję oraz ton odpowiedzi. Po kliknięciu przycisku "Generuj Odpowiedź" dane są wysyłane do serwera. Skrypt JavaScript dba o dynamiczne dostosowanie wysokości pola tekstowego oraz animację załadowania strony. Wykorzystane są zewnętrzne biblioteki CSS do poprawienia wyglądu i GSAP do animacji.

- Widok panelu użytkownika (rys. 6)



The screenshot shows a web browser window with the title 'Generuj Odpowiedź - Korporacja'. The address bar shows the file path 'C:/Users/pawel/Desktop/Quick_responder/zip/templates/index.html'. The main content area displays a form titled 'GENERUJ ODPOWIEDŹ NA E-MAIL'. The form includes a text input field for 'Treść e-maila' with placeholder text 'Wklej treść e-maila'. Below this are two sections for selecting response characteristics: 'Charakter odpowiedzi:' with radio buttons for 'Pozytywna' (selected) and 'Negatywna', and 'Wybierz ton odpowiedzi:' with radio buttons for 'Formalny' (selected) and 'Nieformalny'. A blue button labeled 'Generuj Odpowiedź' is positioned below these options. At the bottom, there is a section labeled 'Wygenerowana odpowiedź:' followed by a large, empty text area for the generated response.

Rysunek 6 Frontend programu

- Przykład użycia (rys. 7)

✉ Treść e-maila:

Temat: Propozycja współpracy – Rozwój wspólnego projektu. Szanowni Państwo, mam przyjemność zwrócić się z propozycją nawiązania współpracy pomiędzy naszymi firmami. Jesteśmy przekonani, że połączenie naszych zasobów i doświadczeń w branży mogłoby przynieść obopólne korzyści i przyczynić się do rozwoju innowacyjnych produktów. Chciałbym zaprosić Państwa do rozmowy na temat szczegółów oraz wspólnych możliwości. Proszę o kontakt, aby umówić się na spotkanie lub rozmowę telefoniczną w dogodnym dla Państwa terminie. Z poważaniem, Adam Kowalski, Business Development Manager

☑ Charakter odpowiedzi:

☒ Pozytywna ☐ Negatywna

👤 Wybierz ton odpowiedzi:

☒ Formalny ☐ Nieformalny

⚙️ Generuj Odpowiedź

↩ Wygenerowana odpowiedź:

Temat: Re: Propozycja współpracy – Rozwój wspólnego projektu

Szanowny Panie Adamie Kowalski,

Dziękujemy za przesłaną propozycję współpracy oraz za uznanie potencjału synergii pomiędzy naszymi firmami. Z zainteresowaniem zapoznaliśmy się z przedstawioną koncepcją rozwoju wspólnego projektu i jesteśmy otwarci na dalsze rozmowy.

Z przyjemnością przyjmujemy zaproszenie do spotkania w celu omówienia szczegółów i możliwości współpracy. Proszę o podanie kilku propozycji terminów dogodnych dla Pana. Chętnie dostosujemy się do Państwa dostępności.

Z poważaniem,

[Imię i nazwisko]
[Nazwa firmy]
[Tytuł stanowiska]
[Dane kontaktowe]

Rysunek 7 Działanie programu

- Logi serwera Flask (rys. 8)

```
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.97.117:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 768-599-605
127.0.0.1 - - [15/Jan/2025 16:01:37] "OPTIONS /generate_reply HTTP/1.1" 200 -
127.0.0.1 - - [15/Jan/2025 16:01:39] "POST /generate_reply HTTP/1.1" 200 -
127.0.0.1 - - [15/Jan/2025 16:02:28] "OPTIONS /generate_reply HTTP/1.1" 200 -
127.0.0.1 - - [15/Jan/2025 16:02:30] "POST /generate_reply HTTP/1.1" 200 -
127.0.0.1 - - [15/Jan/2025 16:05:07] "OPTIONS /generate_reply HTTP/1.1" 200 -
127.0.0.1 - - [15/Jan/2025 16:05:09] "POST /generate_reply HTTP/1.1" 200 -
```

Rysunek 8 Widok konsoli serwera python

W logach serwera Flask widoczny jest ruch sieciowy związany z obsługą żądań pochodzących od frontendu działającego na localhost. Logi pokazują szczegółowe informacje, takie jak adres IP klienta, datę i godzinę każdego żądania, a także rodzaje zapytań HTTP, w tym OPTIONS i POST.

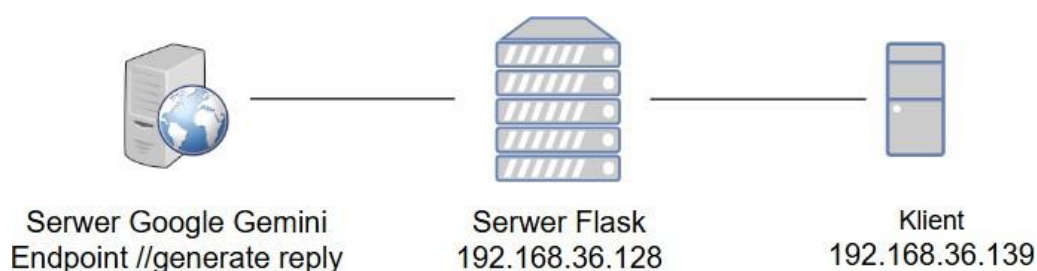
Zapytania typu OPTIONS dotyczą protokołu CORS i służą do sprawdzenia, czy frontend ma odpowiednie uprawnienia do komunikacji z backendem. Z kolei zapytania POST wskazują na przesyłanie danych od użytkownika lub aplikacji frontendowej do serwera.

4.3 Tworzenie topologii komunikacji między klientem, serwerem Flask i modelem Google Gemini

Aplikacja, którą stworzyliśmy, składa się z dwóch głównych elementów: serwera oraz interfejsu użytkownika (frontend). Początkowo oba te komponenty działały lokalnie, na maszynie, na której zostały uruchomione, co nie w pełni odzwierciedlało potencjał systemu. Wykorzystując bibliotekę Flask, stworzyliśmy bardziej zaawansowaną topologię, w której klient, działający na jednym urządzeniu, łączy się z serwerem uruchomionym na innym hoście.

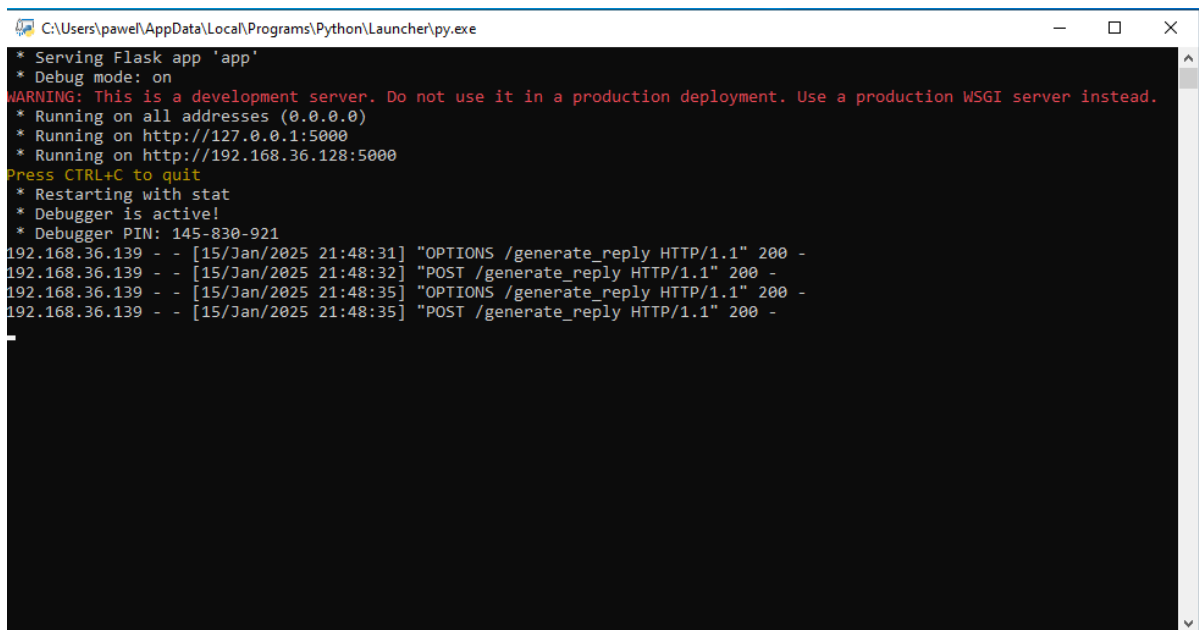
W tym celu fizycznie połączyliśmy dwa komputery przewodem sieciowym. Jeden z nich pełnił rolę serwera, na którym uruchomiony był serwer Flask, natomiast drugi komputer działał jako host, czyli urządzenie, na którym znajdował się interfejs użytkownika (frontend).

W tej konfiguracji komputer hosta wysyłał zapytania do serwera Flask, który odbierał je, przetwarzał i zwracał odpowiedzi. W celu prawidłowego działania systemu, w kodzie frontendowym zmieniliśmy domyślny adres IP (127.0.0.1, czyli localhost) na adres IP serwera (192.168.36.128), co pozwoliło na komunikację pomiędzy oboma komputerami. Poniżej znajduje się schemat topologii sieciowej (rys. 9).



Rysunek 9 Schemat topologii sieciowej

- Logi serwera Flask (rys. 10)



```
C:\Users\pawel\AppData\Local\Programs\Python\Launcher\py.exe
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.36.128:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 145-830-921
192.168.36.139 - - [15/Jan/2025 21:48:31] "OPTIONS /generate_reply HTTP/1.1" 200 -
192.168.36.139 - - [15/Jan/2025 21:48:32] "POST /generate_reply HTTP/1.1" 200 -
192.168.36.139 - - [15/Jan/2025 21:48:35] "OPTIONS /generate_reply HTTP/1.1" 200 -
192.168.36.139 - - [15/Jan/2025 21:48:35] "POST /generate_reply HTTP/1.1" 200 -
```

Rysunek 10 Widok zapytań w konsoli serwera python

5. Podsumowanie oraz określenie dalszych kierunków rozwoju aplikacji

Stworzona przez nas aplikacja stanowi podstawę do jej dalszego rozwoju. Zastosowane w niej narzędzia dają możliwość rozbudowania funkcjonalności oraz zwiększenia łatwości użytkowania. Poniżej znajdują się propozycje rozwoju aplikacji.

- Wdrożenie dodatku do różnych klientów pocztowych takich jak Gmail lub Outlook.
- Dodanie opcji generowania odpowiedzi na wiadomości e-mail różnego typu – nie tylko o charakterze biznesowym.
- Rozbudowanie interfejsu użytkownika o kolejne możliwości dostosowania wygenerowanej odpowiedzi (długość odpowiedzi, jej zawartość, formatowanie tekstu).
- Wzbogacenie aplikacji o funkcje automatyzacji, takie jak planowanie odpowiedzi, wysyłanie przypomnień czy analiza priorytetów wiadomości e-mail.

5.1 Podsumowanie

Nasz projekt, który polegał na stworzeniu aplikacji wykorzystującej generatywne modele językowe do automatycznego generowania odpowiedzi na wiadomości e-mail, dał nam cenne doświadczenie w zakresie tworzenia aplikacji i integracji nowoczesnych technologii. Dzięki temu procesowi zgłębiliśmy zagadnienia związane z programowaniem, integracją API oraz tworzeniem aplikacji webowych.

W trakcie realizacji projektu napotkaliśmy liczne wyzwania, które pozwoliły nam lepiej zrozumieć, jak działają współczesne narzędzia i jak rozwiązywać problemy techniczne.

Projekt dał nam również szerszą perspektywę na wykorzystanie generatywnych modeli językowych w automatyzacji komunikacji elektronicznej. Udowodniliśmy, że takie narzędzia mogą znacząco wspierać procesy biznesowe.

6. Bibliografia

- <https://developer.thunderbird.net/add-ons/about-add-ons>
- <https://support.mozilla.org/pl/kb/thunderbird-dodatki-faq>
- <https://boringowl.io/tag/flask>
- <https://qualityisland.pl/postman-co-to-jest-jak-zainstalowac-i-uzywac-to-narzedzie/>
- <https://jakwybrachosting.pl/cors/>
- <https://pl.wikipedia.org/wiki/XAMPP>
- <https://ai.google.dev/api?hl=pl&lang=python>
- <https://platform.openai.com/docs/guides/text-generation>